Backend Engineering Task

Stelloid Command Center Dashboard API

Owner: Pushpendra Jaiswal | Assignee: Shailesh | Due: 14 October 2026

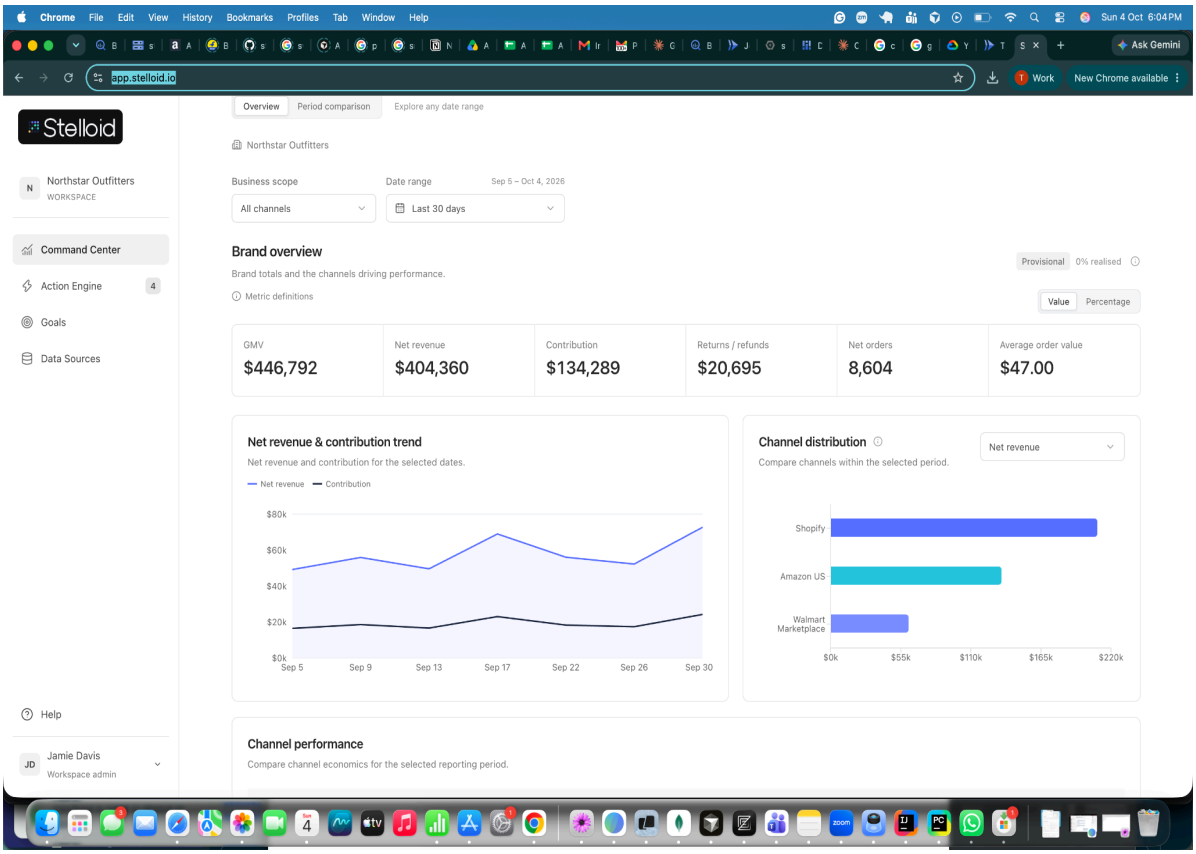
1. Objective

Build a backend service using FastAPI or Django REST Framework that provides mock analytics data to support the Command Center dashboard shown in the reference screenshot. The backend should store the mock dataset in a database and expose clean REST APIs that a frontend can consume.

2. Dashboard to Support

Use the supplied screenshot as the UI reference. The API should support the following visible dashboard concepts:

- Brand overview: GMV, net revenue, contribution, returns/refunds, net orders, and average order value.
- Net revenue and contribution trend over a selected reporting period.
- Channel distribution by metric, with channels such as Shopify, Amazon US, and Walmart Marketplace.
- Channel performance for the selected reporting period.
- Business scope/channel filtering and date-range selection.



3. Scope of Work

- Choose either FastAPI or Django REST Framework. FastAPI is preferred for a lightweight REST service, but either is acceptable.
- Use a relational database such as PostgreSQL or SQLite for the mock dataset.
- Create database models/tables for channels, daily metrics, and any additional dimensions needed to answer the API queries.
- Seed the database with realistic synthetic data for multiple dates and channels. Do not use real customer or company data.
- Implement date-range and channel/business-scope filters.
- Return JSON responses with stable field names and sensible error responses for invalid requests.
- Provide automated tests for the main API endpoints.
- Provide a README with setup, migration, seed, run, and API usage instructions.

4. Suggested Data Model

A simple normalized model is sufficient. You may adjust the design if you can justify it.

Entity	Key fields	Purpose	Example
channels	id, name, business_scope	Available sales channels	shopify, amazon_us, walmart_marketplace
daily_channel_metrics	date, channel_id, gmv, net_revenue, contribution, returns_refunds, net_orders	Daily channel-level measures	2026-10-01 + Shopify
daily_brand_metrics	date, gmv, net_revenue, contribution, returns_refunds, net_orders	Daily totals used for overview/trend	2026-10-01 totals

5. Required API Endpoints

Method	Endpoint	Purpose	Minimum response
GET	/api/v1/overview	Summary cards for a selected date range	GMV, net revenue, contribution, returns/refunds, net orders, average order value
GET	/api/v1/trends	Time-series trend for selected dates	date, net_revenue, contribution
GET	/api/v1/channel-distribution	Channel comparison	channel, metric, value
GET	/api/v1/channel-performance	Channel-level economics	channel + the core performance metrics
GET	/api/v1/channels	List channels	id, name, business_scope

Filter parameters: start_date, end_date, channel, business_scope, metric.

6. Example Response Shape

```
{
  "date_range": {"start": "2026-09-05", "end": "2026-10-04"},
  "overview": {
    "gmv": 446792,
    "net_revenue": 404360,
    "contribution": 134289,
    "returns_refunds": 20695,
    "net_orders": 8604,
```

```
    "average_order_value": 47.00
  }
}
```

7. Mock Data Requirements

- Populate at least 7 consecutive days of data.
- Populate at least 3 channels, including Shopify, Amazon US, and Walmart Marketplace.
- Make the numbers internally consistent where practical. For example, average order value can be derived from GMV/net orders.
- Support different business/channel selections so the frontend can demonstrate filtering.
- Include enough variation across dates and channels to make the trend and distribution charts meaningful.
- Provide a repeatable seed command/script so the database can be rebuilt easily.

8. Acceptance Criteria

- A fresh developer can clone/open the project, configure the database, run migrations, seed mock data, and start the API using the README.
- All required endpoints return valid JSON and support the documented filters.
- The responses are sufficient to populate the dashboard sections shown in the screenshot without hardcoded frontend values.
- At least 5 meaningful API tests are included, covering overview, trends, channel distribution, filtering, and an invalid request.
- No real or confidential customer/company data is included.
- Code is reasonably structured, readable, and documented.
- The completed backend is shared with Pushpendra for review by 14 October 2026.

9. Optional Stretch Goals

- Swagger/OpenAPI documentation with clear examples.
- Docker Compose setup for API + database.
- Pagination/caching where useful.
- A small health endpoint such as GET /health.
- Seed generation that produces deterministic data from a fixed random seed.

10. NDA / Confidentiality

This task is based on an internal product/dashboard reference. By receiving or working on this task, the assignee must treat all supplied screenshots, requirements, data models, API details, credentials, business information, and related materials as confidential.

- Do not share, publish, upload, or disclose the task materials or any derived internal information outside the authorized project team.
- Use only synthetic/mock data in the implementation. Do not copy real customer, account, transaction, or production data into the project.
- Do not commit secrets, credentials, access tokens, private URLs, or other sensitive information to source control.
- Use the information only for completing and reviewing this assigned task.
- Return or delete local copies of confidential materials when requested by the project owner.

Note: These confidentiality terms are task-level instructions and do not replace any formal company NDA, internship agreement, employment agreement, or security policy that may already apply.

11. Timeline

Date	Milestone	Expected outcome
8-9 Oct 2026	Setup + data model	Project skeleton, DB schema, initial seed data
10-11 Oct 2026	Core APIs	Overview, trends, channels, filters
12-13 Oct 2026	Tests + cleanup	API tests, validation, README, seed reproducibility
14 Oct 2026	Final delivery	Review-ready backend shared with Pushpendra

12. Delivery

Owner: Pushpendra Jaiswal

Assignee: Shailesh

Final delivery date: 14 October 2026

Primary deliverable: backend repository/project with database schema, mock data seed, REST APIs, tests, and README.